



End-to-End Testing for AI Infrastructure

Validate the **entire path**: data → features → model → serving → SLOs → logs/audit

Catch **integration gaps** unit tests miss

Produce a **Go/No-Go report** for Week 52

What E2E Testing Covers

Pipelines

Ingestion → validation →
feature store → training
→ registry

Serving

Gateway/mesh →
inference → feature
lookups → storage

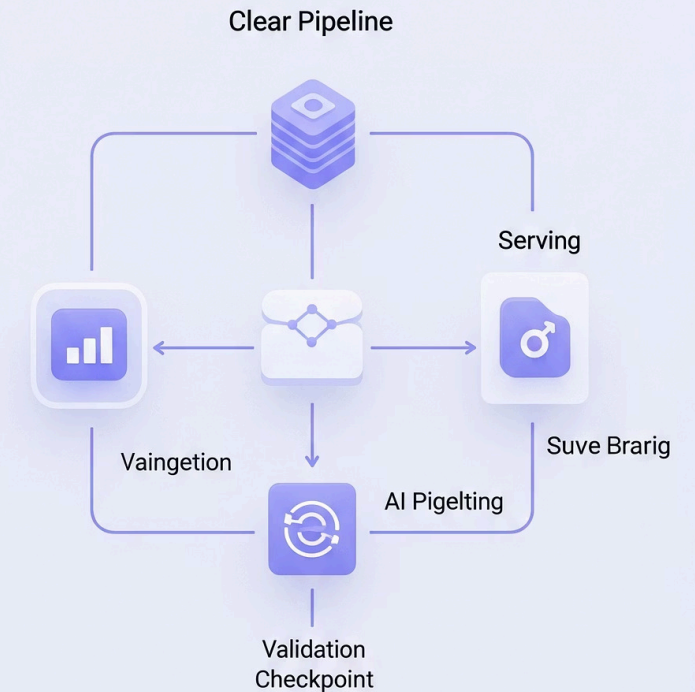
Non-functionals

Latency, availability, cost,
security, compliance

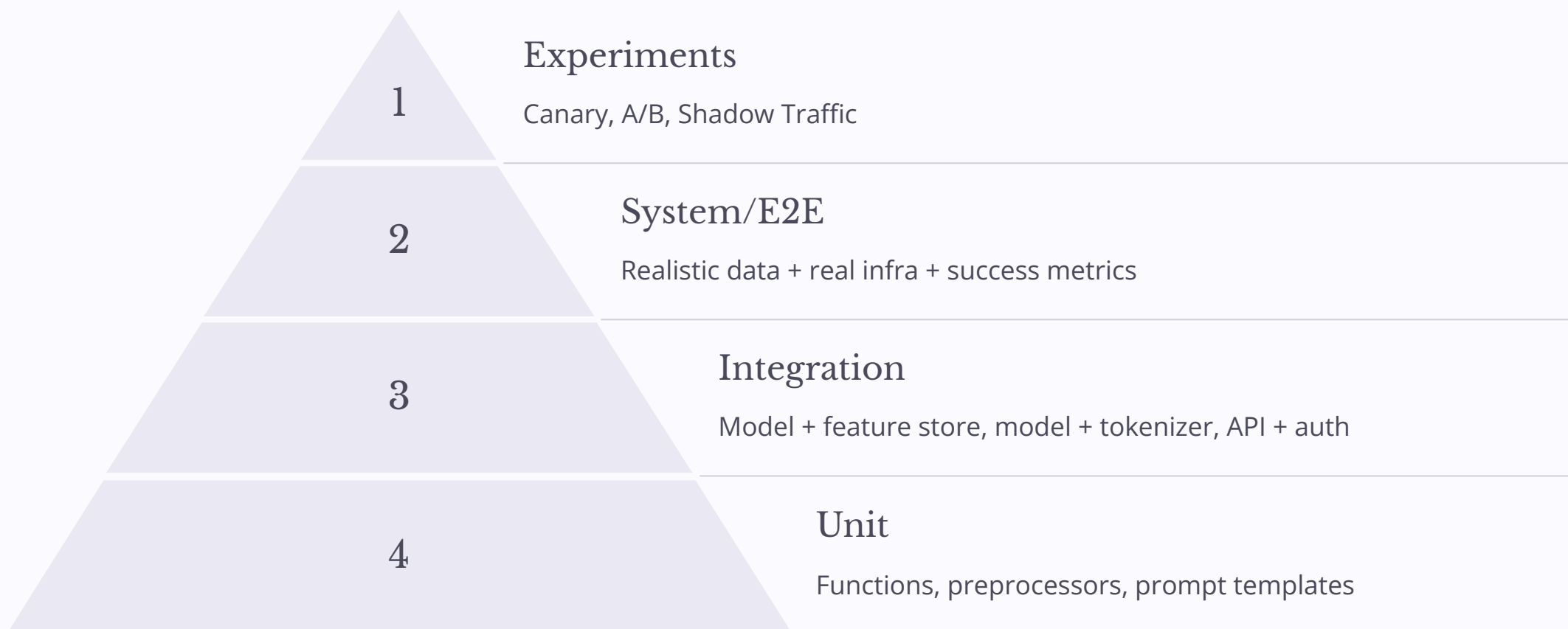
Runbooks

Alerts, rollback, retrain
triggers

AI igeltione



The AI Testing Pyramid



E2E testing validates the [complete system integration](#) that unit tests can't cover

E2E Success Criteria (Define First)

Performance

p95 latency & error rate within SLO

Functional

Correctness on **golden dataset**

Cost

Guardrails (\$/1k req, GPU util %) respected

Compliance

Audit, PII handling, access controls pass

Recovery

Rollback/retrain works and is reproducible

Environment Strategy

Prod-like staging

Same configs, smaller scale

Ephemeral test namespaces

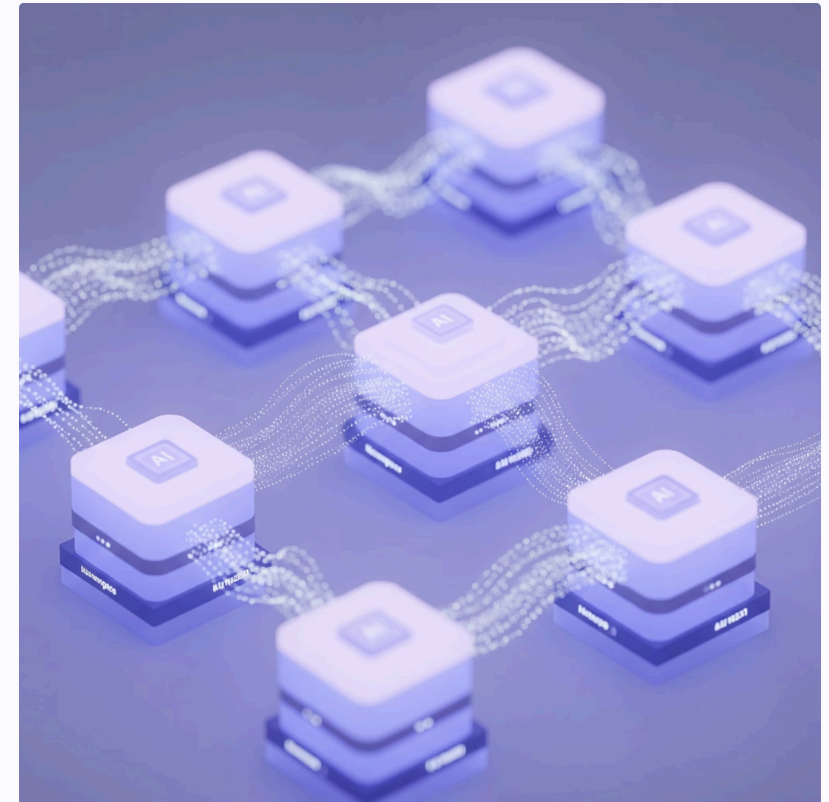
One per PR/branch

Deterministic seeds

Pinned artifacts (model, data snapshot)

Isolated test accounts

Dedicated service accounts & quotas



Golden Test Data & Seeds

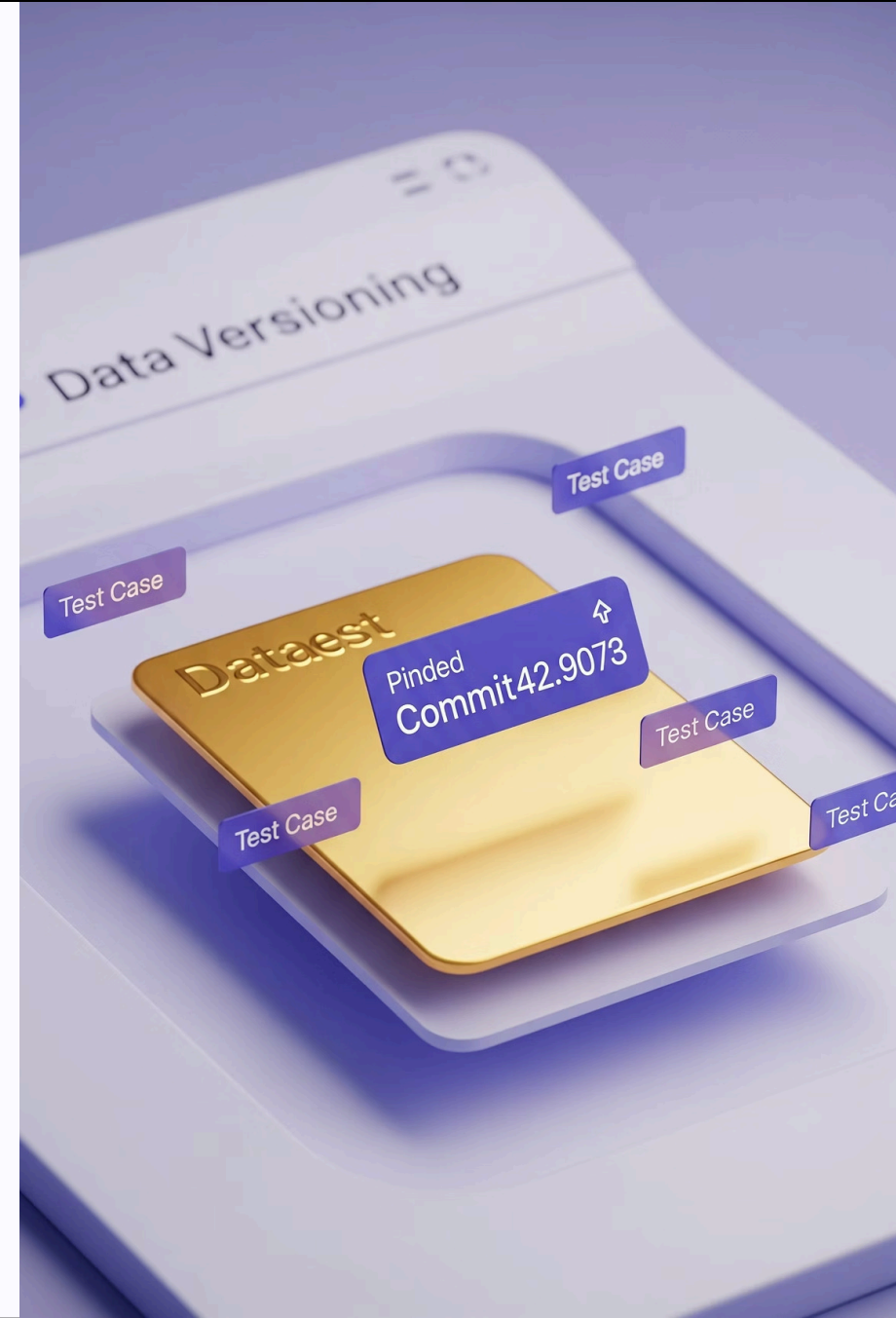
For reproducible, deterministic testing of AI systems:

- Build **golden dataset** with common & edge cases
- Pin dataset with [DVC/LakeFS](#) commit hash
- Control randomness with seeds in all frameworks
- Store **expected outputs** or tolerance bands

```
# Control randomness across frameworks
import torch
import numpy as np
import random
import tensorflow as tf
```

SEED = 42

```
random.seed(SEED)
np.random.seed(SEED)
torch.manual_seed(SEED)
tf.random.set_seed(SEED)
```



Data Pipeline E2E Testing



Ingest

Raw data sources



Validate

Great Expectations/Deequ checks



Compute

Feature transformations



Materialize

Feature store

Key Assertions

- Schema validation
- Value ranges & distributions
- Missingness patterns
- Join keys integrity
- Data freshness
- [Offline vs online feature parity](#)

```
# Example Great Expectations checkpoint (YAML)
expectations:
- expect_column_values_to_not_be_null: {column: user_id}
- expect_column_values_to_be_between: {column: age, min_value: 0, max_value: 120}
```

Kick off training job

Fixed seed for reproducibility

Validate metrics

Min accuracy / max loss thresholds

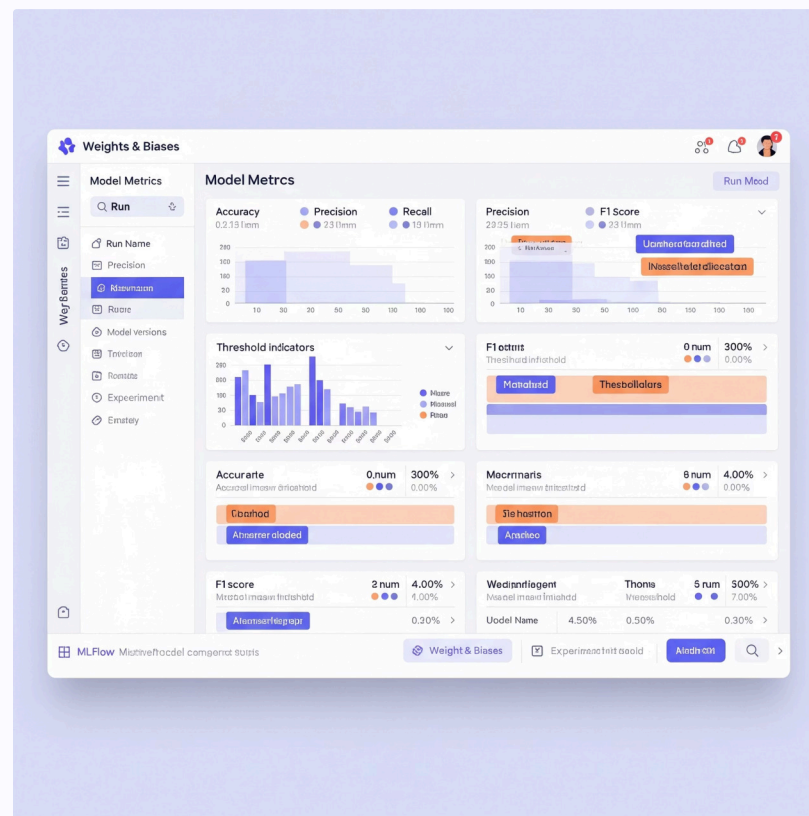
Artifact sizes & model lineage

Promote to STAGING

Only if all metric gates are met

Verify metadata

Model card & audit trail complete



Serving Path E2E Testing (Happy Path)



Validate: auth accepted, 200 OK responses, schema conformance, p95 latency

Ensure correlation IDs persist across all service hops

Pytest Smoke Test Example

```
import requests, time

def test_basic_prediction():
    t0 = time.time()
    r = requests.post(
        BASE+"/predict",
        json={"text":"hello"}
    )

    # Status check
    assert r.status_code == 200

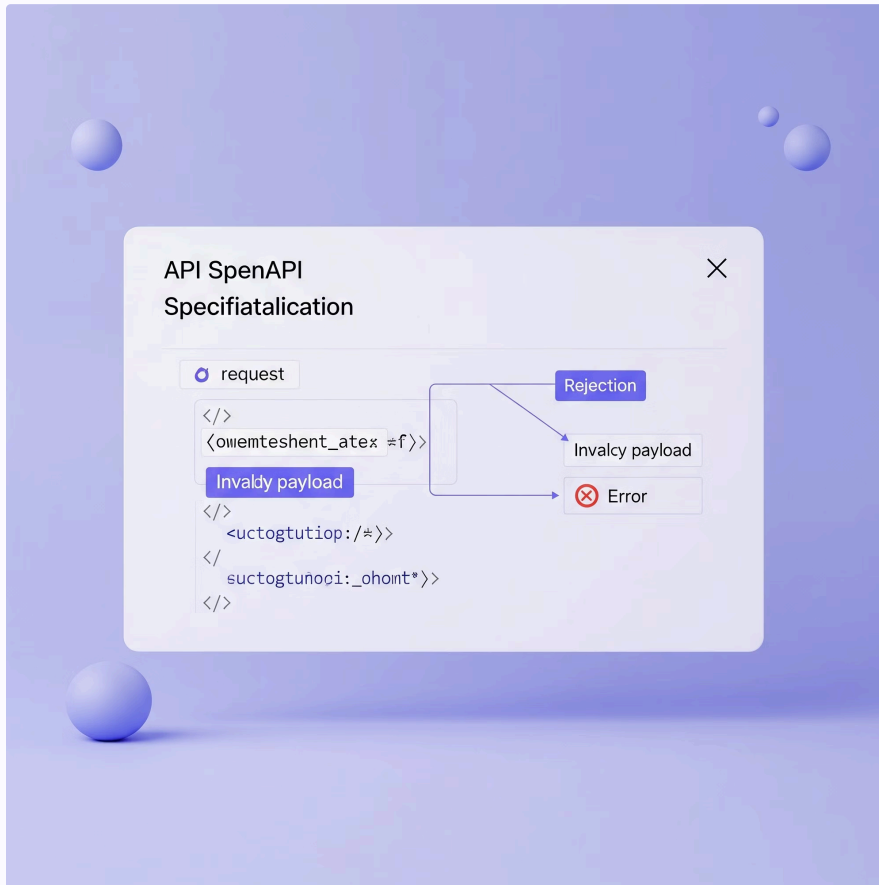
    # Latency check
    lat = (time.time()-t0)*1000
    assert lat < 300

    # Schema check
    assert "label" in r.json()
```

Contract & Schema Testing

Validation Strategy

- Validate API against [OpenAPI](#) specification
- Reject malformed payloads with clear error messages
- Ensure backward-compatibility for clients
- Test deprecation paths
- Use property-based fuzzing to test:
 - Field length limits
 - Unicode handling
 - Oversized payloads



Negative & Edge Case Testing



Input Variations

- Empty inputs
- Massive payloads
- Rare category values
- Out-of-distribution data



System Conditions

- Request timeouts
- Dependency failures
- Slow feature store
- Network partitions



Auth Scenarios

- Rate-limit exceeded
- Expired tokens
- Missing scopes
- Incorrect permissions

Expect **graceful degradation** + actionable error messages in all cases

Performance & Soak Testing

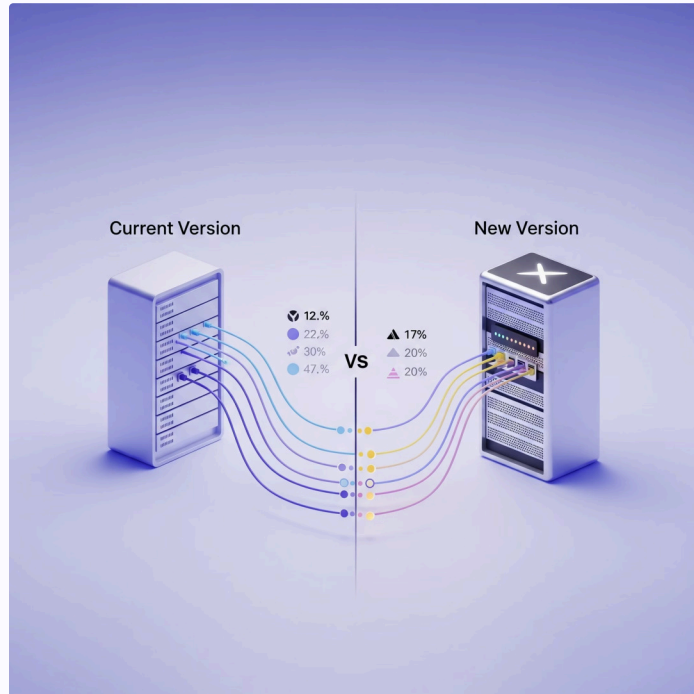
Methodology

- Use [k6/Locust](#) scripts that mirror production traffic mix
- Track multiple dimensions concurrently:
 - p95/p99 latency percentiles
 - Error rates across services
 - Cost per 1,000 requests
 - GPU utilization & memory patterns
- Run 2-4 hour soak tests to detect:
 - Memory leaks
 - File handle exhaustion
 - GPU temperature issues



Canary & Shadow Validation

Canary Testing



Send N% traffic to [vNext](#); compare win-rate & latency deltas

Shadow Testing

1

Duplicate Requests

Copy production traffic to new model

2

Zero User Impact

Responses discarded, only metrics collected

3

Statistical Analysis

Requires **significant improvement** or tie + cheaper

Security & IAM E2E Testing

AuthZ Matrix

Test user/service identities against scopes/roles for allowed/denied access

mTLS Validation

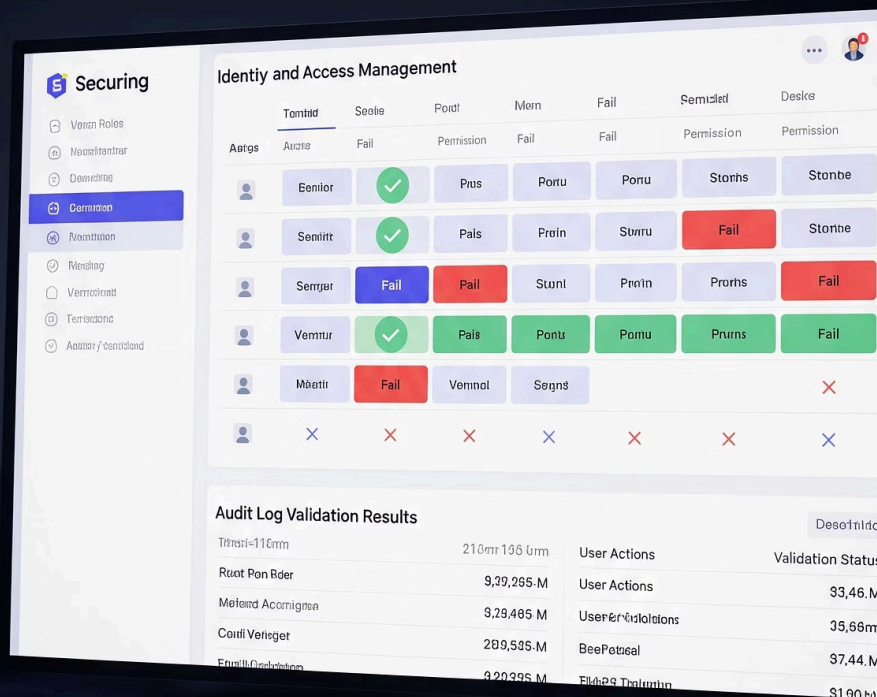
Verify mutual TLS/SPIFEE identity between services

Secret Management

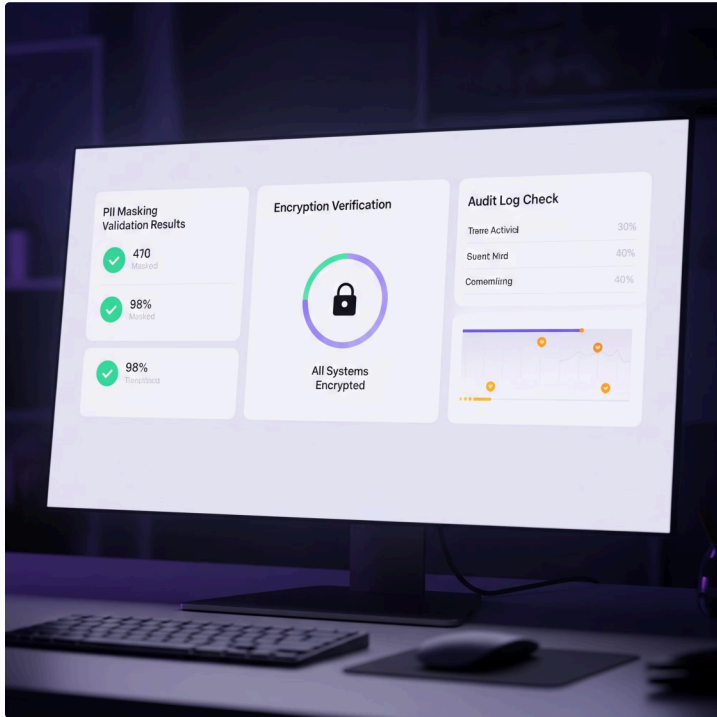
Access via KMS/Vault only; no plaintext environment variables

Audit Trail

Log **who did what** (nonrepudiation)



Compliance & Audit E2E Testing



Critical Compliance Checks

- **PII handling:** Verify masking/tokenization of sensitive data
- **Encryption:** Validate at rest and in transit protection
- **Data residency:** Confirm geographic processing constraints
- **Audit logs:** Ensure immutable WORM storage with complete trace IDs
- **Safety filters** (for GenAI): Test refusal patterns with reason codes

Observability Under Test

Health Probes

Test readiness/liveness probe behavior during deployments

Alert Verification

Create real alerts for p95 latency, 5xx errors, drift detection, and Kafka lag

Dashboard Validation

Ensure release markers are visible and runbook links resolve correctly





Reproducibility & Rollback Testing

Training Reproducibility

- Re-run training with identical seeds
- Verify metrics within defined tolerance

Rollback Drill

- Flip to previous model via registry
- Measure recovery time against SLO

Recovery Metrics

- Record RTO (Recovery Time Objective)
- Measure RPO (Recovery Point Objective)

E2E Automation in CI/CD

Workflow Components

- Spin up [ephemeral namespace](#) for each PR
 - Deploy complete system to isolated environment
 - Run comprehensive E2E test suite
 - Collect all test artifacts and metrics
- Gate merge on passing:
 - SLO requirements
 - Contract tests
 - Data quality checks
 - Security scans
 - Upload artifacts for analysis:
 - k6 JSON results
 - Grafana snapshots
 - Model evaluation metrics

Example Ephemeral Deploy (Helm)

```
# Create unique namespace for this PR
NS=e2e-$GITHUB_SHA
kubectl create ns $NS

# Deploy application to ephemeral namespace
helm upgrade --install app charts/app -n $NS \
  --set image.tag=$GITHUB_SHA \
  --set ingress.host=app-$GITHUB_SHA.local

# Run E2E tests against isolated deployment
pytest -q tests/e2e --base "https://app-$GITHUB_SHA.local"

# Clean up resources after tests complete
kubectl delete ns $NS --wait
```

This approach provides [complete isolation](#) for testing each PR without interfering with other environments



E2E Test Reporting Template

1

Environment

Cluster/node types, model version, data hash

2

Functional Tests

Pass/fail matrix (happy/edge/negative cases)

3

SLO Metrics

p50/p95/p99 latency, error%, cost/1k vs targets

4

Security & Compliance

IAM matrix results, audit log verification

5

Issues & Owners

Root cause + fix plan + assigned owner

6

Final Decision

Go / No-Go (with canary deployment plan)

Common E2E Pitfalls & Fixes

Flaky Tests

Fix: Adjust timeouts, seed randomness, warm caches

Feature Store Drift

Fix: Implement feature store parity checks

Green Tests, Red SLOs

Fix: Include load testing in E2E, not just functional checks

Audit Failures

Fix: Add compliance assertions in pipeline



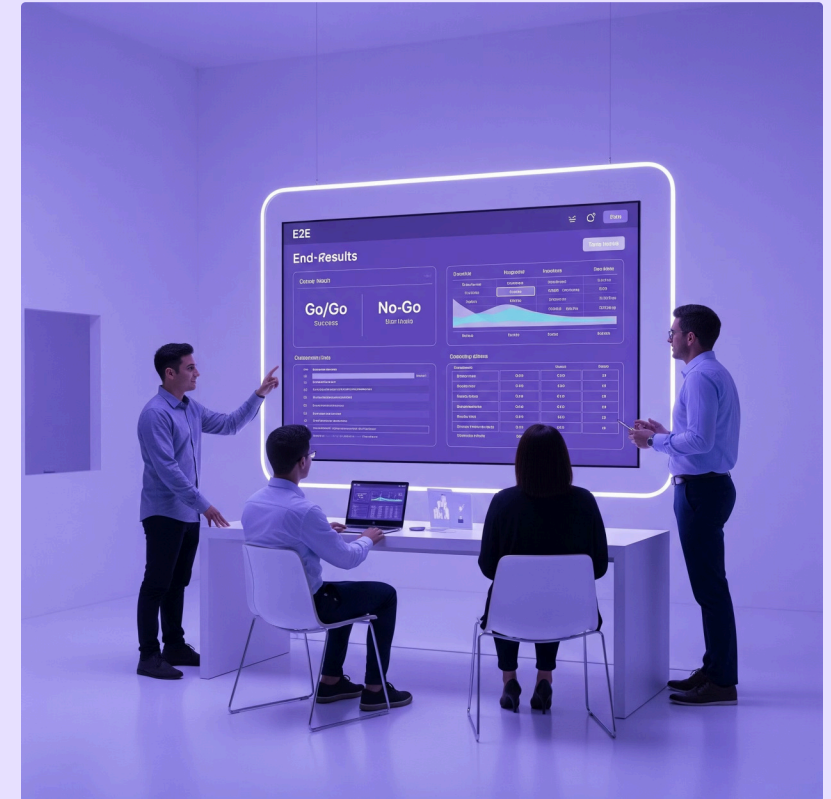
Summary

End-to-End testing provides **evidence** that your AI infrastructure meets:

- Functional correctness requirements
- SLO performance targets
- Compliance obligations

Test **complete system integration**:

- Pipelines + serving + security + observability
- Automate in CI/CD with ephemeral, prod-like environments
- Produce clear **Go/No-Go** report for Week 52





Discussion

Which E2E gate would most likely block your release today?



Latency SLO



Auth/Compliance



Data Parity

What's your plan to fix it?